Anderson Contreira
Posted on Nov 28, 2025

Edit Manage Stats

Finding Balance in Architectural Decisions

#architecture #development

In software projects, not every decision needs a full-blown architectural committee or a 30-page design document. But the opposite is also true: some decisions are too important to be treated as a simple quick fix.

Every engineering team faces moments where they need to choose between doing something the right way or doing something fast. In Brazil, we often call the fast solution a gambiarra or puxadinho, an improvised workaround that solves the problem for now but usually comes with long-term costs. In English, the closest terms would be a patch, hacky fix, temporary workaround, or band-aid solution.

And the reality is: sometimes you do need a quick workaround to move forward. Deadlines exist, business pressure is real, and teams often need short-term progress. The problem starts when these quick fixes become the foundation of the system.

Not Every Decision Needs Heavy Architecture

But some absolutely do.

Good architects and senior engineers know how to find this balance. They know when something can be done fast and when it needs a more structured approach. They ask questions like:

Is this decision easy or painful to reverse later?
Will this create technical debt that affects core flows?
Are we introducing hidden coupling that will slow us down?
Is this a workaround, or are we building a long-term solution?

This balance is what separates sustainable architecture from a collection of short-lived patches.

Workarounds Have a Purpose, but They Also Have a Cost

A workaround can help the product move forward, validate an idea, or unblock a team. The issue is that workarounds tend to accumulate quietly. One temporary decision becomes another. That small puxadinho added today becomes tomorrow's bottleneck.

Over time, the system becomes fragile, harder to test, difficult to refactor, and resistant to change. What was once a shortcut becomes a long-term liability.

How to Make Better Architectural Decisions

You don't need to over-engineer everything. But you do need awareness, intention, and clarity. Some principles help:

1. Understand what is hard to change.
   Database schemas, communication patterns, module boundaries, and API contracts deserve careful thought.

2. Keep documented trade-offs.
   Tools like ADRs (Architecture Decision Records) help future teams understand why a decision was made.

3. Separate workarounds from architecture.
   A workaround should always be labeled, documented, and revisited later.

4. Design for adaptability.
   Loosely coupled components, clear interfaces, and simple boundaries reduce long-term pain.

5. Consider the business context.
   Sometimes the right decision is the fast one, as long as everyone understands the cost.

Final Thoughts

Architecture is not about choosing the perfect pattern or technology. It is about balance, trade-offs, timing, and intention. It is knowing when a gambiarra is enough to move forward and when it is time to slow down and build the right foundation.

In the end, architecture is a continuous conversation, not a single decision. The systems we build reflect the choices we make along the way. Make them intentionally.

Top comments (0)

Code of Conduct • Report abuse

Anderson Contreira

Solutions Architect. IT professional since 2008. Over 6 years of leadership experience. Specialist in distributed architecture, microservices, cloud (AWS/GCP), DevOps and Full-Stack experience.

LOCATION
Brazil

EDUCATION
Distributed Software Architecture @ PUC Minas

WORK
Cloud Software Archtect @ Caylent

JOINED
Oct 31, 2025

More from Anderson Contreira

WARNING TO DEVELOPERS: A new wave of “technical test scams” is targeting devs

#development #scam #recruiting

ALERTA IMPORTANTE PARA DEVS: golpe em testes técnicos está crescendo - fiquem espertos

#braziliandevs #scam #recruiting #development

Quick tip for anyone building a career in Tech

#career #development